
There are 4 problems totaling 99 points + 1 point for taking the exam:

Attendance: 1 point
Problem 1: 26 points
Problem 2: 24 points
Problem 3: 34 points
Problem 4: 15 points

Assume the following unless noted otherwise:

All programs are built and run on 64-bit Linux, where `sizeof(int)` is 4 and `sizeof(int *)` is 8.

All standard library calls are successful when they are invoked correctly. For example, you can assume that proper `malloc()` calls will succeed.

Code shown on the exam may omit the needed `#include` statements. Assume they are there unless otherwise specified.

If this exam refers to lab code, assume the versions provided by the class, i.e., skeleton code and solutions.

Avoid using hard-coded numbers if possible. Hard-coded numbers make your program error prone and less portable. For example, using `"sizeof(int *)"` instead of `"8"` makes it applicable for both 32-bit and 64-bit environments.

What to hand in and what to keep:

- At the end of the exam, you will hand in only the answer sheet, which is the last two pages (one sheet printed double-sided) of the exam booklet.
- Make sure you write your name & UNI on both sides of the answer sheet.
- All other pages (i.e., the rest of this exam booklet and any scratch papers you have used during the exam) are yours to keep.
- Please write only your final answers on the answer sheet. Verbosity will only hurt your grade. If we find multiple answers to a question, we will cherry-pick the part that will result in the LOWEST grade. This policy ensures that a shotgun approach to solving a problem is never rewarded. Make sure to cross out clearly anything you don't consider your final answer.

PLEASE DO NOT OPEN THIS EXAM BOOKLET UNTIL YOU ARE TOLD TO DO SO!

This exam is for SECTION 001.
You MUST be registered for this section to take this exam.

Problem [2]: 24 points

Consider the following function:

```
// Returns 1 when invoked on a little-endian machine
// Returns 0 when invoked on a big-endian machine
int is_little_endian();
```

(2.1)-(2.6) present six possible implementations of the function, each of which may or may not be correct. Answer each one in the following way:

- Write GOOD if it works as described above.
- Write BAD, followed by two numbers, if it doesn't work as described above, the first number being the return value on a little-endian machine and the second number being the return value on a big-endian machine.

For example, write "BAD 5,7" if you think the function returns 5 on a little-endian machine, and 7 on a big-endian machine. Write "BAD 0,0" if you think it returns 0 regardless of the endianness.

- Write UNPREDICTABLE if the return value can vary from run to run on a single machine

```
(2.1)  int is_little_endian() {
        int x = 1;
        if (x & 0xff000000)
            return 1;
        else
            return 0;
    }
```

```
(2.2)  int is_little_endian() {
        short x = (1 << 8);
        return x;
    }
```

```
(2.3)  int is_little_endian() {
        int x = 1;
        return x != htonl(x);
    }
```

```
(2.4)  int is_little_endian() {
        int x = -1;
        return x != htonl(x);
    }
```

```
(2.5)  int is_little_endian() {
        int x = 1;
        return *(char *)&x;
    }
```

```
(2.6)  int is_little_endian() {
        // 2^15 is 32768
        int x = 12345;
        short y = 12345;
        return *(short *)&x == y;
    }
```

UNI:

Name:

[1]

(1.1) Command on KLAC:

```
(1.2) int main(int argc, char **argv)
{
    assert(argc == 2); // requires one command-line argument
    char c;

    while (read( _____ ) == 1) {

        if ( _____ ) {
            c = toupper(c);
        }

        write( _____ );
    }
}
```

[2]

(2.1)

(2.2)

(2.3)

(2.4)

(2.5)

(2.6)